

Calculator built with a Decentralized Autonomous Architecture Oriented to Services

Roberto Requejo Fernández
Facultad de Ingeniería
Universidad Panamericana
0236851@up.edu.mx

Abstract - This document has the purpose of presenting the development of a calculator with the implementation of a Decentralized Autonomous Architecture Oriented to the Services (ADSOA), where considerations from previous architectures such as Service autonomous decentralized (ADS) and Service Oriented Architecture (SOA) and its application in the JAVA programming language. Likewise, the importance of distributed computing is described, the necessary change to this computing model and the proposals that have occurred in the area.

1. Introduction

The rapid advancement of technology has led to the exploration of innovative approaches to computing and system implementation. This exploration has focused on not only enhancing resilience to failures but also on fostering interconnectivity. To address these needs, distributed computing has emerged as a pivotal solution. Distributed computing systems comprise multiple software components spread across different computers but function as a unified system.

Distributed computing aims to decentralize processes and resources, encompassing not only physical hardware elements but also delivering constant access and scalability. This approach is increasingly relevant in today's interconnected world, where many applications operate seamlessly without centralized servers, making server failures inconspicuous and system performance improvements achievable through distributed implementations.

1.1. Service-Oriented Architecture

To achieve this goal, there are multiple architectures that focus on the primary objective of not centralizing information. Among these, there is the Service-Oriented Architecture (SOA), which is a type of software design that enables the reuse of its elements through service interfaces that communicate over a network with a common language. While the definition may seem theoretical and technical, it relies on a very important word that will be further explored in this writing: reuse. It is possible to approach the definition from another perspective, by separating its

elements in such a way that they can be called upon when needed. This independence and autonomy allow scalability and growth without compromising the performance of the system in which it is implemented.

Before the use of this architecture, adding new features or connecting an application to the services of another was complicated. The centralized model, known as "monolithic," limited the implementation of new functionalities, updates, or corrections to existing ones. If something did not work properly, the entire system had to be shut down, the error resolved, and then restarted as a new version. This seems unthinkable today. Critical systems, those that must be highly reliable and maintain this reliability as the system evolves without incurring prohibitive costs [2], have grown significantly in importance. They are observed in energy sources such as nuclear power plants, where a small error could lead to a catastrophe, as well as in transportation systems that were once considered utopian.

2. Problem

The problem at hand lies in the development of a calculator that implements an Autonomous Decentralized Service-Oriented Architecture (ADSOA). While it may seem like an excessive implementation due to the use of a server to process the operation.

3. Development

To solve this problem, we thought of making three programs. Described as it follows:

1. **Client:** This is a program that will load a friendly user interface capable of making basic operations such as addition, subtraction, division, and multiplication.

2. **Middleware:** This is message-oriented middleware (MOM) server that is uncharged of connecting everything and broadcast the packages to all clients and servers connected.

3. **Server:** The purpose of this program is to receive the packages and process de mathematical operations and send them back for the client to show to the user.

4. First Delivery

This first delivery consists of the three basic programs developed so that we get a basic idea of the program but not fully functional. For the communication between these three programs, we used the TCP protocol through the java sockets library, the communication between programs can be exemplified in the next diagram

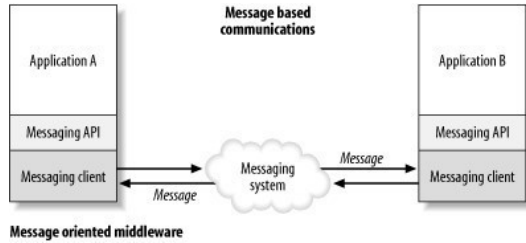


Fig 1. Application communication diagram

For the communication between the three programs, I developed a simple protocol for each program to do what is expected of them, since we broadcast all the packages the programs receive information, they don't use at all. The package I send contains for elements each separated by a coma, it looks like this: "First Element, Second Element, Third Element, Forth Element, Fifth Element". The first element contains whether a mathematical operation has been resolved or is pending to resolve, we use number codes. If the code is 100 means it is pending to resolve and if the code is 200 means it has been resolved. The second and third element is both operands, they are the numbers entered by the user. The fourth element is the type of mathematical operation, and it is represented by the symbol, the options are: +, -, * and /. And finally, the fifth element can be empty, because there we saved the result of the operation, so if the first element is 200, we know the fifth element is not going to be empty. The middleware and the server have no graphic interface, but the client has, and it look like the following figure.



Fig 2. Calculator Interface

Described as it follows: Also here is a demonstration of my program functioning through images.

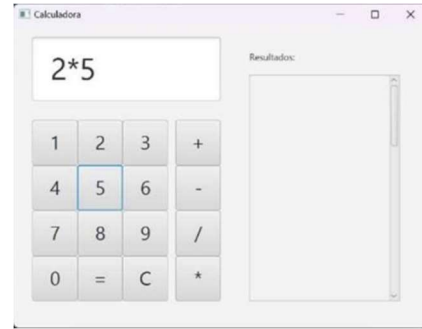


Fig 3. Example of user sending a request

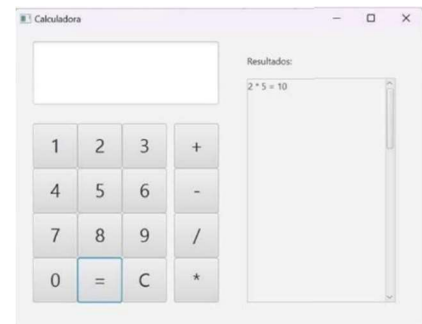


Fig 4. Example response in the client

For this first delivery we can get a good idea of what is the goal of this project, but it has a lot of weak point until now. The main issue is that the middleware broadcast the message to everyone, and a lot of equal packages are generated, meaning that if we have more than one server open the client will receive more than one result and will display them all on the results bar. Also, if there is more than one client open it will display tall the result, even if it didn't request them. This is the main issue of this project as of now.

5. Second Delivery

Well, with this initial approach, the previously stated problem is simply shifted to the middleware. Although what was described in the previous point is relatively less susceptible to failures and crashes due to the presence of multiple servers and clients capable of performing the operation, if the intermediary fails or goes down, it will create an issue with the interconnection, rendering the redundancy with servers and clients useless, as messages won't be passed between their elements. Keeping this premise in mind, it is not unreasonable to consider the inclusion of multiple nodes to mitigate the aforementioned error and introduce redundancy in the connections. Building on this newly stated premise, the importance segmentation of operations is introduced. The significance or criticality of each operation is determined by the number of servers that must respond for the operation to be considered successful. This consideration considers that in a real implementation,

there are services requiring higher priority or greater redundancy based on their functions. Hence, the inclusion of an acknowledgment function becomes important, indicating who has responded and what the minimum required is.

This problematic was solved by modifying the middleware and instead of having one, we have a many nodes that are interconnected to each other. Also, when a client is connected to this network it searches a random node to connect to. This way if one node or server fails it doesn't matter because we have a network where many other servers and cells are working together. The network can be represented like the following figure.

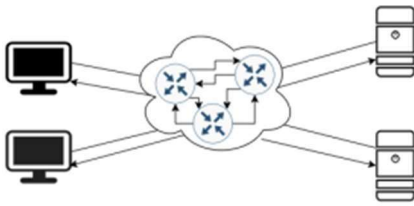


Fig 5. Visual representation of the environment

Also, the way broadcast works changed drastically from the last delivery, last time when the middleware received a package it broadcasted it to everyone, but now that we have many middlewares that are interconnected this will make the program loop in a sea of broadcast that will saturate the system. So, we implemented a new communication program that consists of knowing where the message comes from. If the message comes from a node, it will only broadcast the information to the cells connected, but not other nodes. But if the message comes from a cell, it will be broadcasted to everyone. This is essential to avoid the system getting saturated.

To make the system more reliable we divided the server in four programs that focus on individual operations, one for addition, subtraction, division, and multiplication. We have many instances of these programs connected to random nodes each time one of these programs starts. So, if an addition server fails we have another instance running on another node.

6. Final Delivery

For this ultimate delivery, our paramount objective was to fortify our distributed architecture, ensuring an unparalleled level of resilience. In pursuit of this goal, we implemented two pivotal protocols: the Foliation Protocol and the Dynamic *.jar* File Upload. These additions were strategically designed to enhance the robustness of our system significantly.

The Foliation Protocol introduces a systematic approach to manage and organize information flow within our distributed architecture. By incorporating this protocol, we aim to streamline communication and optimize the

efficiency of data transmission, thereby contributing to the overall reliability of our system.

Additionally, the integration of the Dynamic *.jar* File Upload feature marks a paradigm shift in the way we handle configuration modifications. This innovative approach allows us to dynamically alter the behavior of individual cells by simply updating a *.conf* file, eliminating the need for manual code modifications. This not only enhances flexibility but also promotes ease of maintenance, crucial for a resilient distributed architecture.

Furthermore, a critical improvement involved rectifying an error in the system. Specifically, we ensured that calculators requesting operations now receive a singular result, mitigating any potential confusion or complications arising from multiple outputs. This refinement not only aligns with precision in results but also contributes to a smoother user experience.

Addressing another vital aspect, we implemented a fix to ensure that calculators only receive operations they explicitly requested. This enhancement prevents unintended operations from being processed by calculators, thereby fortifying the security and accuracy of our distributed computing environment.

Foliation Protocol

This protocol, designed to streamline the flow of information within a system, incorporates a sophisticated queuing mechanism for managing both incoming and outgoing data packages. The outbound data package processing follows a meticulously orchestrated sequence:

1. Package Creation and Queue Addition:
 - The process initiates with the creation of a data package that encapsulates the necessary information.
 - Subsequently, this newly generated package is added to the outgoing queue, awaiting its turn for further processing.
2. Queue Administration and Middleware Interaction:
 - The queue administrator takes charge by sending the initial package to the middleware, a central component in the system's architecture.
 - Upon dispatch, the administrator patiently awaits responses from the middleware, signaling the commencement of the data processing cycle.
3. Server Cell Interaction and Operation Notification:
 - Server cells, responsible for handling computational tasks, receive the incoming package.
 - Before tackling the assigned problem, the server cells promptly dispatch a notification package, signaling their active engagement in the operation.
4. Client Operation Validation and Queue Update:
 - The client, having received the notification, evaluates whether the operation meets the predefined requirements set by the programmer.
 - Only upon meeting these criteria does the client

proceed to remove the operation from the queue, signaling its readiness to receive the processed results.

5. Result Submission and Display:

- With the operation successfully executed, the client forwards the results to the appropriate destination.
- Simultaneously, the client's interface updates to display the received results, ensuring users are promptly informed of the outcome of the processed operation.

This meticulous protocol not only ensures a systematic and organized handling of outgoing data packages but also incorporates safeguards and verification mechanisms at each step, contributing to a reliable and efficient information exchange within the system.

In the following chart we can look at how the new operation protocol codes work and what is their meaning. To this package we also added the following fields clientId, cellId and operation id. The clientId and cellId is a hash made of unique components of the client such as port, Ip and I added a random number t make it even more unique. The operationId is another has unique to each operation so we can identify it easily.

Code	Meaning
1	Addition Operation
2	Subtraction Operation
3	Multiplication Operation
4	Division Operation
5	Result
99	Notification of received

Table 1. Protocol explanation

Dynamic .jar Upload

The project's pivotal feature, the dynamic .jar upload functionality, significantly contributes to elevating the manageability and flexibility of the codebase. The expansion of this feature encompasses several key aspects. In the Jar Creation Process, it involves establishing a streamlined procedure for generating dynamic .jar files from compiled Java classes, implementing automation scripts or build tools for simplicity, and incorporating versioning to track changes effectively. Another crucial facet involves Configuration File Handling, where the structure of the configuration file is enhanced to accommodate diverse customization options. A robust parser is implemented to interpret configuration files, allowing users to specify preferences, settings, and modifications through this file.

The project also emphasizes a Modular Design approach, structuring the codebase into modules that align with specific functionalities or features. This design enables dynamic addition or removal of modules based on user configuration, fostering a modular and extensible architecture. Lastly, the Dynamic Class Modification aspect focuses on developing a mechanism for runtime

modifications of Java classes based on configurations. Reflection or bytecode manipulation libraries such as ASM or Byte Buddy are employed for this purpose, ensuring that class modifications adhere to security best practices.

7. Final Delivery

In summary, this document chronicles the development of a calculator implementing an Autonomous Decentralized Service-Oriented Architecture (ADSOA). The exploration begins with an introduction to the significance of distributed computing, emphasizing the evolution from traditional monolithic architectures to Service-Oriented Architecture (SOA) and its role in achieving scalability and autonomy.

The identified problem revolves around the challenge of implementing ADSOA in a calculator, leading to the development of three key programs: Client, Middleware, and Server. The first delivery showcases the basic structure but reveals weaknesses in broadcast handling. The second delivery addresses these issues by introducing multiple nodes, refining broadcast mechanisms, and enhancing server reliability through individualized operations.

The final delivery marks a significant evolution, introducing the Foliation Protocol for systematic information flow and the Dynamic .jar File Upload feature for flexible codebase management. These additions enhance system resilience, streamline communication, and allow dynamic modification without manual code changes. Critical improvements include refining result handling, ensuring precise client-requested operations, and fortifying overall security.

In conclusion, the journey from a basic calculator to a robust ADSOA implementation demonstrates a commitment to addressing challenges iteratively, resulting in a resilient, flexible, and efficient distributed computing environment. The integration of advanced protocols and features reflects a forward-looking approach to distributed system development, contributing to the broader discourse on decentralized architectures and their applications.

References

- [1] “¿Que’ es la arquitectura orientada a los servicios?”, *Redhat.com*. [En l’ínea]. Disponible en: <https://www.redhat.com/es/topics/cloud-native-apps/what-is-service-oriented-architecture>. [Consultado: 24-oct-2022].
- [2] M. Hinchey and L. Coyle, “Evolving critical systems: a research agenda for computer-based systems.”. University of Limerick, 01-Jan-2010 [En l’ínea]. Disponible en: <https://hdl.handle.net/10344/2085>. [Consultado: 24-Oct-2022]
- [3] “Broadcast”, *Ecured.cu*. [En l’ínea]. Disponible en: <https://www.ecured.cu/Broadcast>. [Consultado: 15-sept-2022].
- [4] IBM Cloud Education, “What is middleware?”, *Ibm.com*, 05-mar-2021. [En l’ínea]. Disponible en: <https://www.ibm.com/cloud/learn/middleware>. [Consultado: 15-sept-2022].
- [5] “IBM documentation”, *Ibm.com*, 19-abr-2021. [En l’ínea]. Disponible en: <https://www.ibm.com/docs/en/txseries/8.2?topic=overview-what-is-distributed-computing>. [Consultado: 15-sept-2022].